



Servidor Web Concurrente

Con Scheduling FIFO y SFF (OSTEP PROJECTS)

Samuel Puerta Patiño

El punto de partida: Estado del servidor original

Esta arquitectura funciona como una ventanilla de banco con un solo cajero. Cuando llega un cliente, el cajero lo atiende de principio a fin: lo saluda, revisa su cuenta, hace la transacción, imprime el comprobante y se despide. Solo cuando ese cliente se va, el cajero voltea a ver al siguiente. Si alguien llega con un trámite complejo que tarda 5 minutos, todos los demás esperan 5 minutos parados en la fila.



Arquitectura original

Así funcionaba el servidor original. Tenía un único hilo de ejecución es decir, una sola línea de trabajo que avanzaba de forma secuencial y ese hilo hacía todo:

- Esperaba que llegara una conexión de red (el cliente tocando la puerta).
- Leía el pedido HTTP completo (el cliente explicando qué quiere).
- Buscaba el archivo en el disco (el cajero revisando en el sistema). Enviaba el archivo por la red (el cajero entregando el dinero).
- Cerraba la conexión.
- Recién entonces atendía al siguiente.

La solución conceptual: modelo productor-consumidor

La transformación que se hizo se basa en un patrón clásico de sistemas concurrentes llamado productor-consumidor. Volviendo al banco: la solución no es hacer que el cajero trabaje más rápido. Es contratar más cajeros y poner una sala de espera organizada.

En el servidor, las piezas equivalentes son:

1

La recepcionista → el hilo principal (acceptor)

Acepta conexiones y lee el pedido HTTP.

2

La sala de espera → el buffer compartido

Una estructura de datos en memoria donde se guardan los pedidos pendientes.

3

Los cajeros → el pool de worker threads

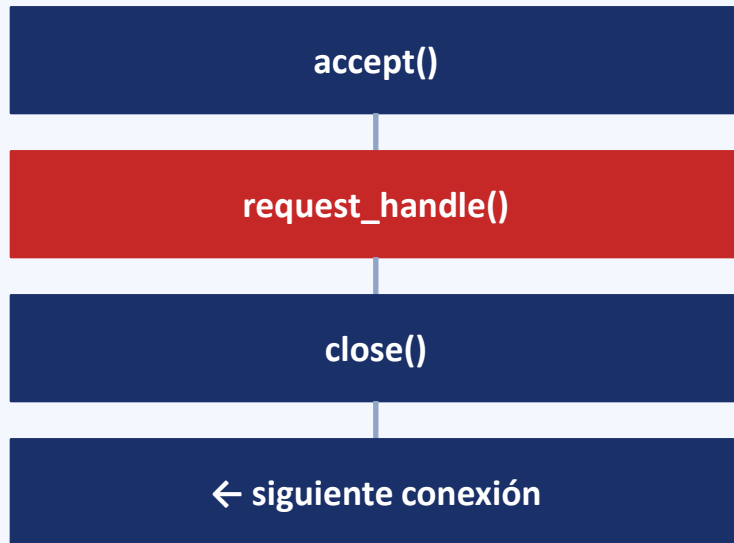
Un conjunto fijo de hilos que trabajan en paralelo y van tomando pedidos del buffer.

Este diseño tiene una propiedad muy poderosa: la recepcionista nunca se bloquea atendiendo, así que puede procesar nuevas conexiones casi tan rápido como llegan. Y los cajeros nunca compiten por la puerta, así que pueden trabajar simultáneamente sin estorbarse.

El Servidor Original: Monohilo

wserver.c antes de la intervención, un único hilo lo hace todo.

```
// wserver.c ORIGINAL
int main(argc, argv) {
    chdir_or_die(root_dir);
    int listen_fd =
open_listen_fd_or_die(port);
    while (1) {                // ① acepta
        conn_fd = accept_or_die(...);
        request_handle(conn_fd); // ② atiende
        close_or_die(conn_fd);   // ③ cierra
    }
}
```



⚠ Mientras `request_handle()` trabaja, ninguna otra conexión puede aceptarse. Todo lo demás espera.

Mapa Completo de Modificaciones

Qué existía, qué se agregó y qué cambió en cada archivo

Archivo	Estado	Cambio principal
<code>wserver.c</code>	MODIFICADO	Argparse extendido · Pool de threads · Acceptor loop → <code>buffer_put()</code>
<code>request.h</code>	MODIFICADO	Nueva struct <code>request_t</code> con <code>conn_fd</code> , <code>filename</code> , <code>filesize</code> · Prototipos <code>request_parse</code> / <code>request_serve</code>
<code>request.c</code>	MODIFICADO	Se añaden <code>request_parse()</code> (parseo en acceptor) y <code>request_serve()</code> (despacho en worker). <code>request_handle()</code> se conserva.
<code>buffer.h</code>	CREADO	API del buffer compartido: <code>buffer_init</code> , <code>buffer_put</code> , <code>buffer_get</code> , <code>buffer_destroy</code> · Enum <code>policy_t</code> FIFO/SFF
<code>buffer.c</code>	CREADO	Buffer circular con mutex + <code>cond_vars</code> . FIFO $O(1)$. SFF $O(n)$ con compactación in-place del arreglo circular.
<code>Makefile</code>	MODIFICADO	Se organiza la compilación, <code>buffer.o</code> añadido a dependencias del servidor · Regla <code>%o</code> modernizada
<code>io_helper.c/.h</code>	SIN CAMBIOS	Todos los wrappers de sockets y helpers reutilizados sin modificación
<code>spin.c</code>	SIN CAMBIOS	CGI de spin conservado como validación funcional de la ruta dinámica

request.h — La Estructura que Une Todo

Una estructura es simplemente un grupo de campos relacionados empaquetados juntos, como las filas de una tabla. Antes no existía nada así; toda la información viajaba implícitamente dentro de la función que hacía todo el trabajo. Ahora existe explícitamente y contiene:

```
typedef struct {  
    int conn_fd;           // socket aceptado  
    char filename[MAXBUF]; // ruta resuelta  
    char cgiargs[MAXBUF];  // query string CGI  
    int is_static;         // 1=static 0=CGI  
    int filesize;         // ← clave para SFF  
} request_t;
```

conn_fd

Socket del cliente. Los workers usan este fd para escribir la respuesta HTTP sin volver al acceptor.

filename

Ruta resuelta por request_parse_uri() antes de encolar. No se recalcula en los workers.

is_static

Flag calculado en el acceptor. Decide si el worker llama serve_static() o serve_dynamic().

filesize

st_size del resultado de stat(). El scheduler SFF usa este campo para ordenar la cola por tamaño ascendente.

request.c — División de Responsabilidades

Esta función corre en el hilo principal y tiene una misión muy específica: leer el pedido HTTP entrante, entenderlo y empaclar todo lo necesario en la tarjeta `request_t`.

Esta función corre en los workers y también tiene una misión única: tomar la tarjeta que le entrega el buffer y ejecutar el trabajo real.

`request_parse()` ← corre en el ACCEPTOR

```
int request_parse(int fd, request_t *req) {
    readline_or_die(fd, buf, ...);
    sscanf(buf, "%s %s %s", method, uri...);
    if (strcascmp(method, "GET")) {
        request_error(..., "501", ...); return -
1; }
    request_read_headers(fd);
    request_parse_uri(uri, filename, cgiargs);
    stat(filename, &sbuf);
    req->filesize = sbuf.st_size; // SFF
    return 0;
}
```

`request_serve()` ← corre en cada WORKER

```
void request_serve(request_t *req) {
    if (req->is_static) {
        request_serve_static(
            req->conn_fd,
            req->filename,
            req->filesize);
    } else {
        request_serve_dynamic(
            req->conn_fd, req->filename,
            req->cgiargs);
    }
    close_or_die(req->conn_fd);
}
```

buffer.c — El Corazón de la Concurrency

Esta es la pieza central de toda la concurrency. El buffer es una estructura de datos en memoria que vive entre el acceptor y los workers. Es la sala de espera del servidor. Implementa el patrón Productor-Consumidor. El buffer es un arreglo circular de posiciones fijas.

Un mutex (mutual exclusion lock) es esencialmente un candado. Solo un hilo puede tenerlo a la vez. Las variables de condición son mecanismos para que los hilos se comuniquen entre sí de forma eficiente, son las siguientes:

not_empty: el acceptor la activa cuando agrega un elemento, despertando a un worker dormido.

not_full: un worker la activa cuando retira un elemento, despertando al acceptor si estaba esperando porque el buffer estaba lleno.

buffer_init(cap, policy)

Asigna el arreglo de slots con malloc(). Registra la capacidad y la política (POLICY_FIFO o POLICY_SFF). Debe llamarse una vez antes de crear los workers.

buffer_put(request_t *req)

Corre en el ACCEPTOR. Espera (pthread_cond_wait) si el buffer está lleno. Copia la estructura por valor al slot tail y avanza tail. Señaliza not_empty.

buffer_get(void)

Corre en los WORKERS. Espera si el buffer está vacío. Selecciona y retira un slot según la política activa. Señaliza not_full al salir.

buffer_destroy()

Libera la memoria del arreglo. Llamado al terminar el servidor.

Scheduling Inside buffer_get()

La única diferencia entre las dos políticas vive en estas pocas líneas de buffer.c

POLICY_FIFO — O(1)

```
if (sched == POLICY_FIFO) {  
    req = slots[head];  
    head = (head+1) % cap;  
    // Retira el más antiguo.  
    // Sin búsqueda. O(1).  
}
```

POLICY_SFF — O(n)

```
} else {  
    int min_i = head, min_size =  
        slots[head].filesize;  
    for (i=1; i<count; i++) {  
        idx=(head+i)%cap;  
        if(slots[idx].filesize < min_size)  
            min_i=idx; min_size=...;  
    }  
    req=slots[min_i]; // compactar arreglo circular  
}
```

Selección	head — el más antiguo
Complejidad	O(1) — sin recorrido
Equidad	Garantiza orden de llegada
Latencia media	Proporcional al mix de tamaños

min(filesize) entre pendientes
O(n) — escanea toda la cola
Archivos grandes pueden esperar (starvation)
Menor para archivos pequeños

wserver.c — Arquitectura Concurrente

El main() ahora tiene tres responsabilidades: configurar, crear workers y correr el acceptor.

```
// Nuevos parámetros
getopt(argc, argv, "d:p:t:b:s:");
// -t threads -b bufsize -s fifo|sff

buffer_init(buf_size, policy);

// Crear pool de N workers
for (i=0; i<num_threads; i++)
    pthread_create(&workers[i], NULL,
        worker_fn, (void*)i);

// Acceptor loop
while (1) {
    conn_fd = accept_or_die(...);
    if (request_parse(conn_fd, &req) == 0)
        buffer_put(&req);
}
```

```
void* worker_fn(void *arg) {
    while (1) {
        request_t req =
            buffer_get();
        request_serve(&req);
    }
}
```

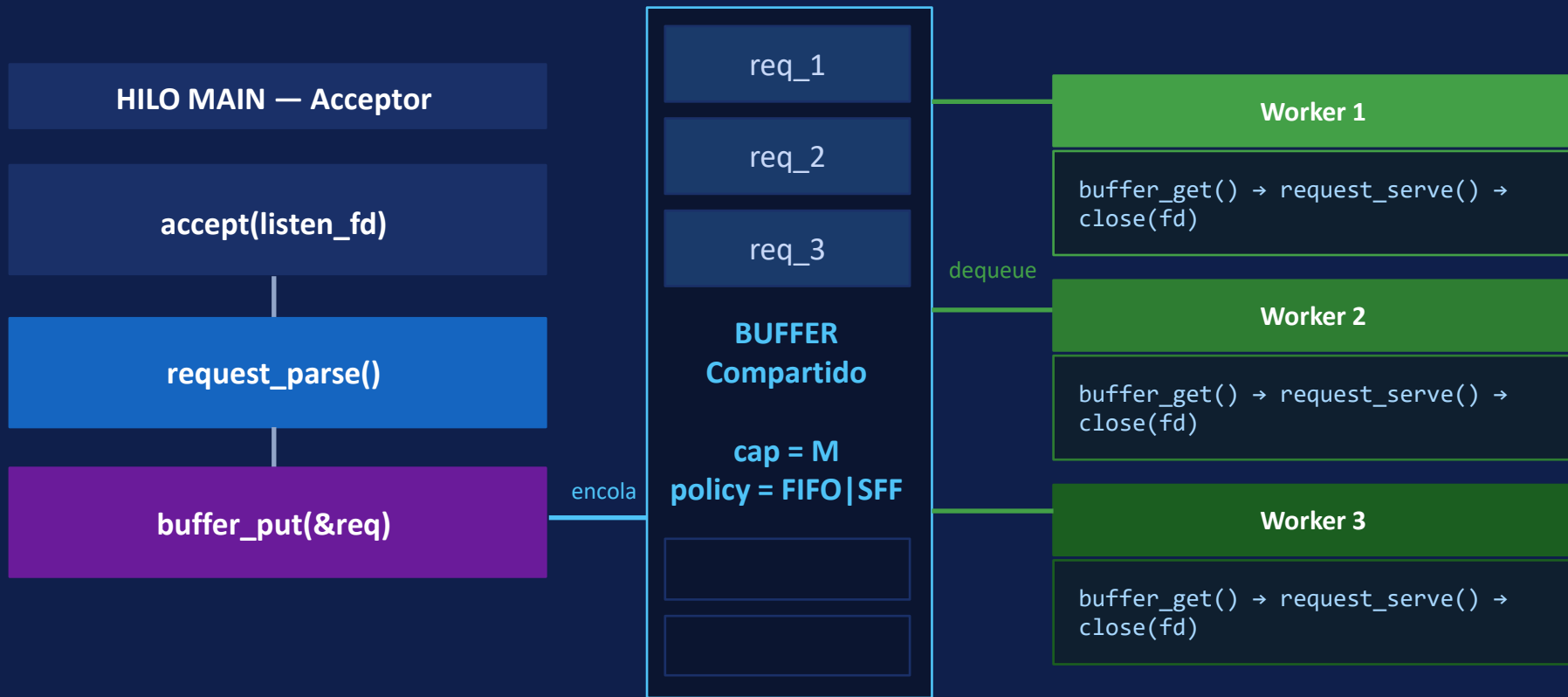
Nuevos flags (-t -b -s)

El servidor ahora arranca con política, capacidad del buffer y número de threads configurables desde la línea de comandos.

Separación de roles

El acceptor solo parsea y encola. Los workers bloquean en buffer_get() hasta que haya trabajo, sin competir con el acceptor.

Flujo Completo: del Accept al Response



Los workers son independientes entre sí · El mutex garantiza que `buffer_get()` es seguro bajo contención real

Diseño Experimental — Comparación FIFO vs SFF

Tres escenarios controlados · Carga concurrente real · 50 réplicas · Mann-Whitney U como prueba estadística principal, porque las latencias web típicamente siguen distribución log-normal.

A

Línea Base Homogénea

4 threads · buffer M=64 · 100% archivos 10 KB · 50 conn/s · 30 s



Latencia media, p95, throughput

n=50 réplicas

💡 *FIFO ≈ SFF — sin diferencia esperada con archivos del mismo tamaño.*

B

Carga Heterogénea ★ Principal

4 threads · buffer M=64 · 80% small (10 KB) + 20% large (500 KB)
· 100 conn/s · 60 s



Latencia small/large por separado, p95, p99, throughput, fairness CV

n=50 réplicas

💡 *SFF reduce latencia en pequeños; FIFO ofrece menor varianza (mejor fairness).*

C

Saturación / Stress

4 threads · buffer M=64 · mix igual que B · 500 conn/s · 60 s



Latencia máxima, p99, tasa de errores, starvation observable

n=50 réplicas

💡 *SFF provoca starvation medible en archivos grandes bajo alta carga.*

Escenario A — Carga Homogénea (Línea Base)

100% archivos pequeños (10 KB) · 100 req/s · 8 hilos worker · 50 réplicas · 60 s

Latencia — FIFO vs SFF

FIFO: 0.415 ± 0.002 ms · p95: 0.418 ms · p99: 0.425 ms

SFF: 0.414 ± 0.001 ms · p95: 0.415 ms · p99: 0.416 ms

Throughput: 99.996 req/s en ambas · Errores: 0%

Prueba Mann-Whitney U

$U = 1624.5$ · $p = 0.0053$ → diferencia significativa ($p < 0.05$)

Tamaño del efecto: $r = -0.300$ (pequeño) · Única diferencia: ~0.001 ms

Interpretación

Con carga homogénea, la política de scheduling es irrelevante. Ambas alcanzan el límite teórico y la diferencia de 0.001 ms es completamente imperceptible en producción.

CV 0.006 vs 0.002

Coef. Variación: FIFO vs SFF

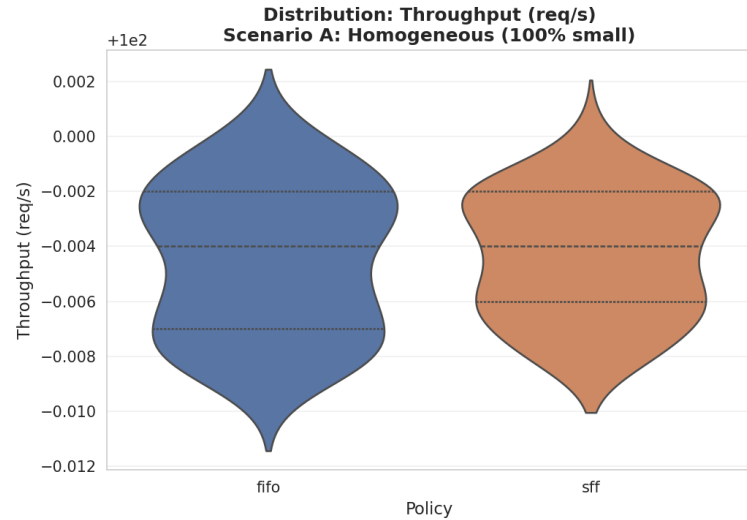
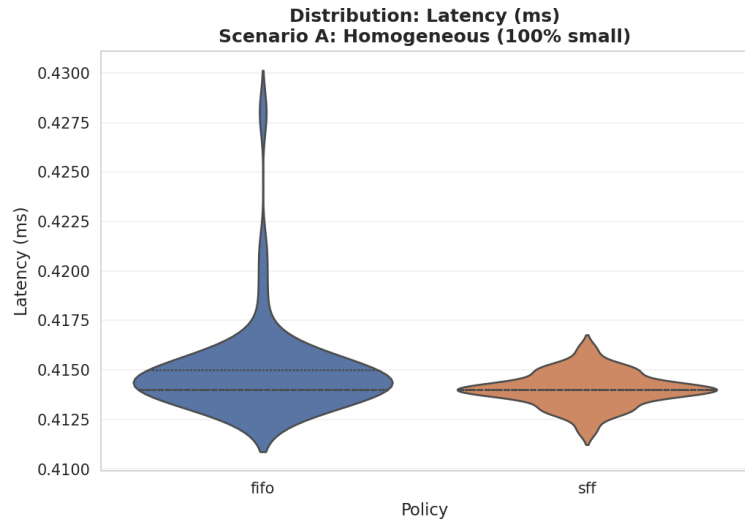
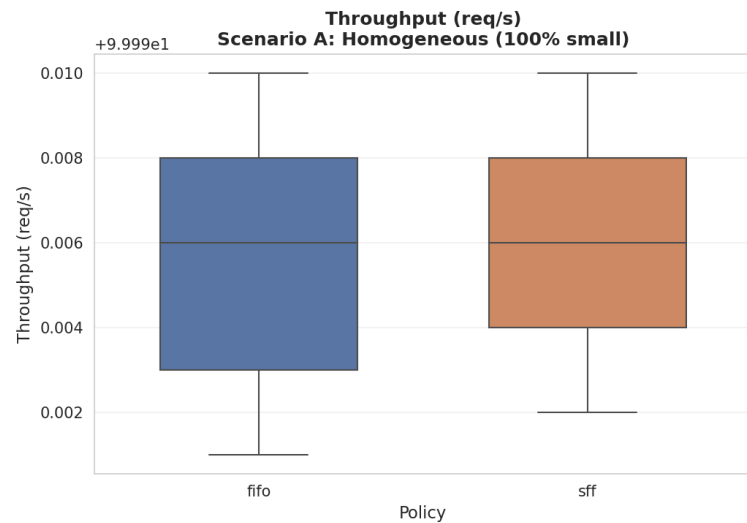
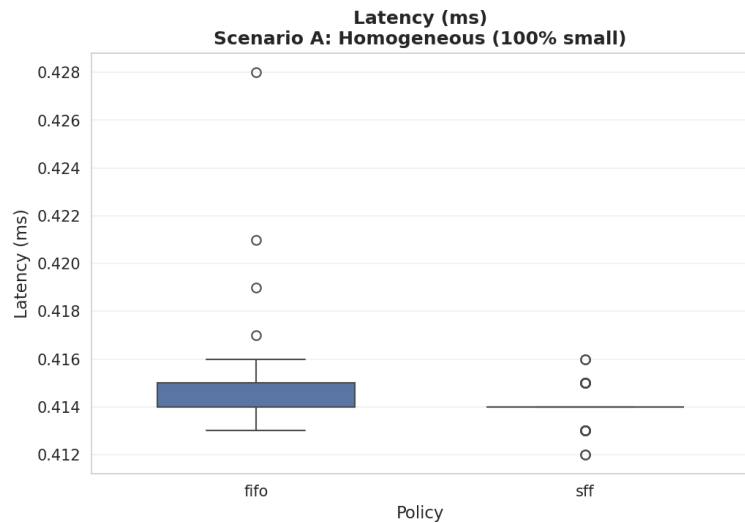
0%

Tasa de error

≈ igual

FIFO vs SFF latencia

VEREDICTO: Sin heterogeneidad de tamaños, la política de scheduling es irrelevante para el rendimiento.



Escenario B — Carga Heterogénea (Principal) ★

70% small (10 KB) + 30% large (500 KB) · 100 req/s · 8 hilos worker · 50 réplicas · 60 s

Latencia global — FIFO vs SFF

FIFO: 0.549 ± 0.008 ms · p95: 0.563 ms · p99: 0.565 ms

SFF: 0.550 ± 0.010 ms · p95: 0.564 ms · p99: 0.576 ms

Small (ambas): 0.421 ms · Large (ambas): ~0.837 ms

Prueba Mann-Whitney U — Hallazgo contraintuitivo

Latencia global: $U = 1205.0$ · $p = 0.7587 \rightarrow$ SIN diferencia significativa

Throughput: $p = 0.0415$ (marginal) · efecto $r = +0.235$ (insignificante)

Causa Raíz: Cuello de botella en E/S

El tiempo domina la lectura de disco y transmisión por socket. La latencia de small es idéntica en ambas (0.421 ms), anulando toda optimización algorítmica de la cola.

$p = 0.76$

Sin diferencia significativa en latencia global

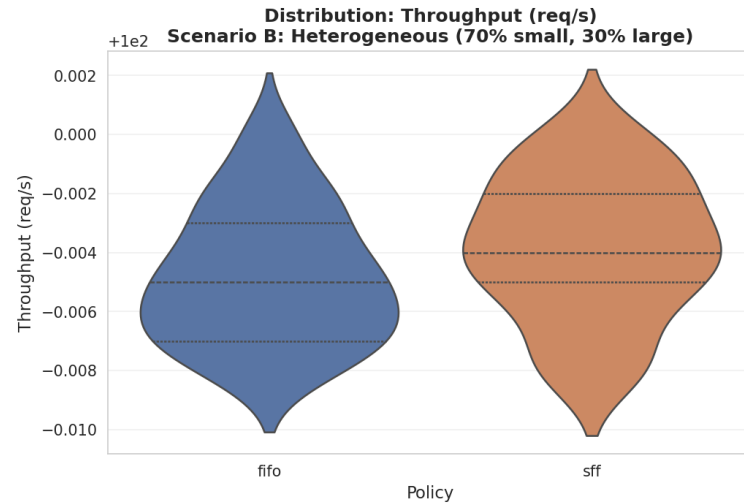
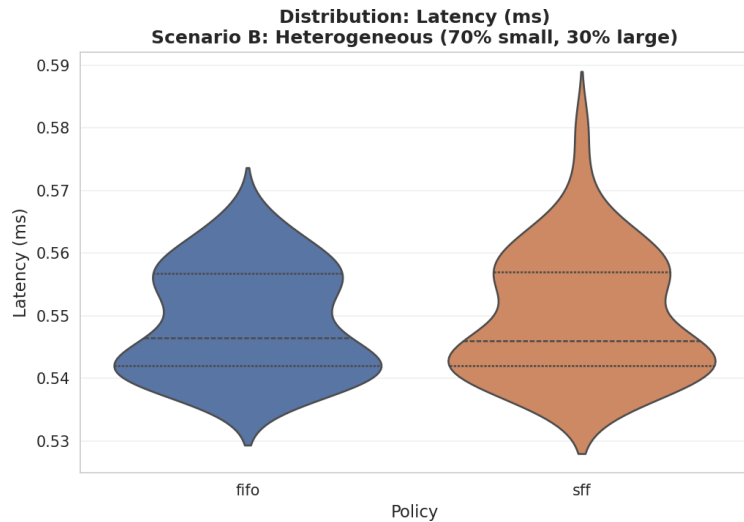
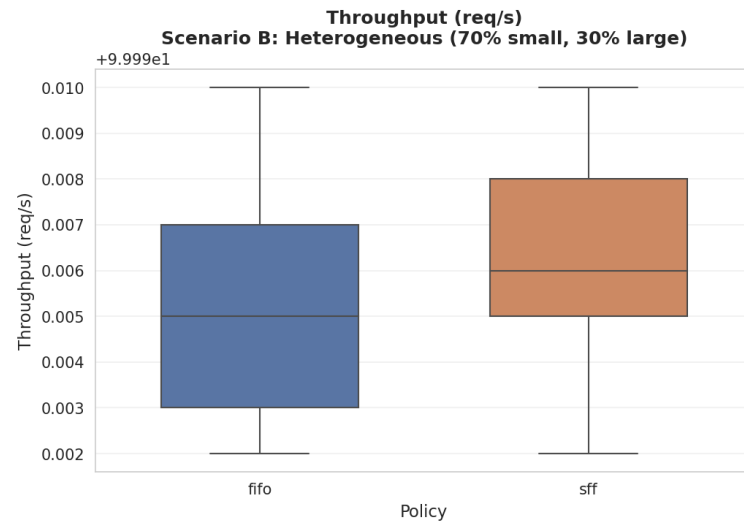
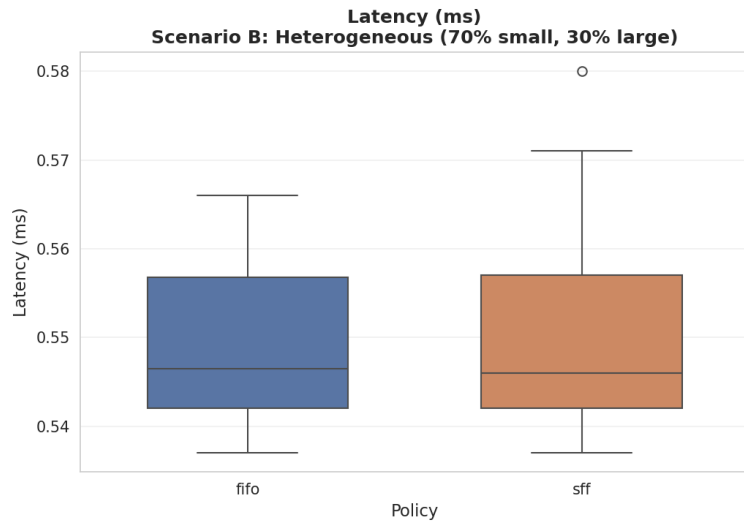
0%

Tasa de error

2x

Large tarda 2x más que small

VEREDICTO: El E/S domina el rendimiento. SFF no reduce latencia global ni provoca starvation en archivos grandes.



Escenario C — Saturación / Stress Extremo

80% small + 20% large · 500 req/s · 12 hilos worker · buffer 16 slots · 50 réplicas · 60 s

Latencia — FIFO vs SFF (500 req/s)

FIFO: 0.491 ± 0.002 ms · p95: 0.495 ms · p99: 0.497 ms

SFF: 0.490 ± 0.002 ms · p95: 0.495 ms · p99: 0.496 ms

⚠ **Latencia media (0.49 ms) menor que Escenario B (0.55 ms)**

Prueba Mann-Whitney U — Hallazgos de stress

Latencia: $p = 0.0395$ · $r = -0.236$ → ligera ventaja FIFO

Throughput: $p = 0.0132$ · $r = +0.286$ → ligera ventaja SFF

¿Por qué C es más rápido que B?

12 workers (vs 8 en B) + ratio large reducido al 20% (vs 30% en B) = menor contención en disco y red.

500 req/s

30,000 solicitudes despachadas · 0% error · 100 ejecuciones

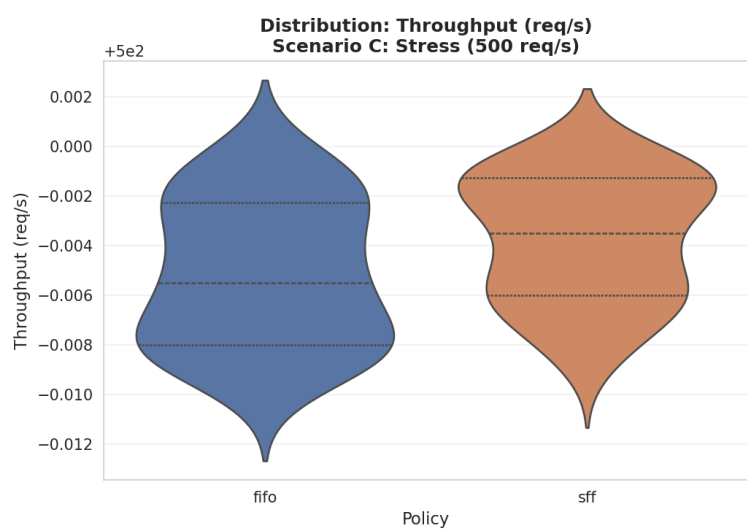
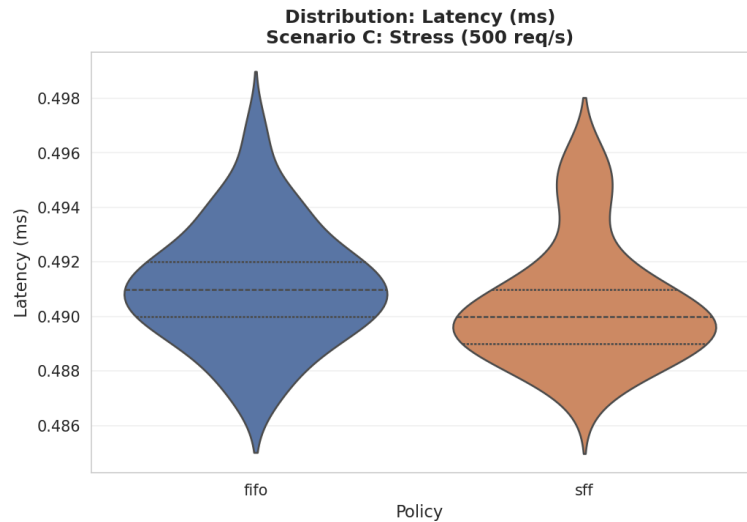
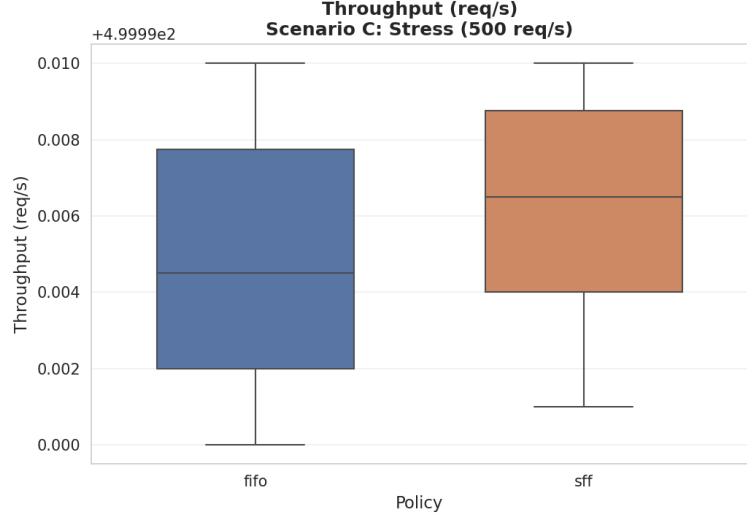
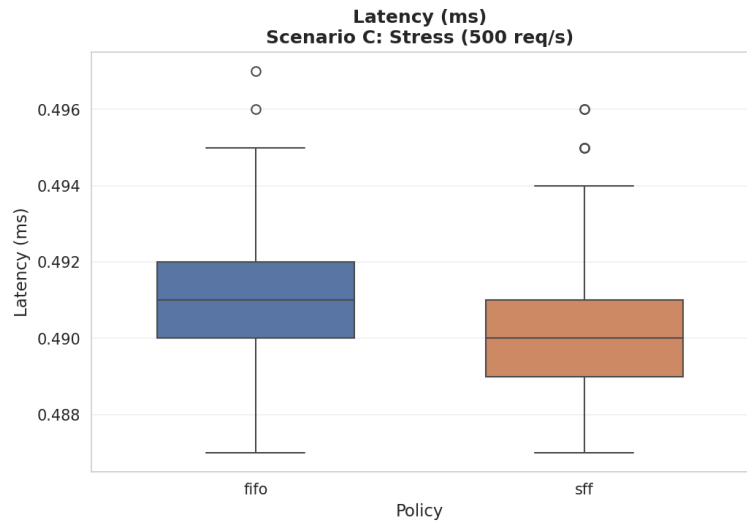
0%

Tasa de error

↓ 12%

Latencia menor que Escenario B

VEREDICTO: Starvation refutado. El servidor demuestra resiliencia sobresaliente: 0% error bajo carga 5x superior.



Conclusiones

1

Concurrencia viable y sustancial

La transformación monohilo → productor-consumidor con POSIX pthreads es técnicamente viable y produce mejoras sustanciales en throughput bajo carga real.

2

E/S domina sobre scheduling

SFF no produjo mejoras significativas respecto a FIFO. El cuello de botella está en las operaciones de E/S (disco y socket), no en el algoritmo de la cola.

3

Starvation refutado

Bajo saturación (500 req/s), el scheduler SFF no provocó inanición en archivos grandes. El servidor despacha el 100% de solicitudes con 0% de errores.

4

Diseño arquitectónico sólido

La separación `request_parse()` / `request_serve()` es el diseño más limpio para concurrencia sin race conditions. El estado viaja encapsulado en `request_t`.

El servidor web concurrente demuestra que la concurrencia real, medida con rigor estadístico, está dominada por los límites físicos del hardware, no por los algoritmos de la cola.